NTT-AI-HPC

Accelerating Number Theoretic Transforms with OpenMP and a Learned Cost Model

R. Abinav
ME23B1004
CS5013 — High Performance Computing

Contents

1 Introduction **2**

2 Theory **2**
2.1 Number Theoretic Transform 2
2.2 NTT in STARK Proofs 2

3 Why a Learned Cost Model **3**

4 Results **4**
4.1 Speedup Curve 4
4.2 Threading Overhead Crossover 4
4.3 MLP Training Loss 5
4.4 MLP: Predicted vs Actual Thread Count 5
4.5 XGBoost Feature Importance 6
4.6 XGBoost: Predicted vs Actual Thread Count 6
4.7 Three-Way Comparison: Serial vs Pure-HPC vs AI-HPC 7
4.8 Key Numbers 7
4.9 STARK Demo Output 8

5 Conclusion **9**

1 Introduction

The Number Theoretic Transform (NTT) is the modular-arithmetic equivalent of the FFT. It operates entirely over integers modulo a prime, producing exact results with no floating-point error. Its primary use is fast polynomial multiplication in $O(n \log n)$ — a hard requirement in modern cryptographic systems.

This project parallelises the NTT with OpenMP and trains a cost model (XGBoost) to select the optimal thread count for any given input size, avoiding the thread-spawn overhead that makes naive parallelism harmful for small inputs.

2 Theory

2.1 Number Theoretic Transform

Given a polynomial $A(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}$, the NTT evaluates it at n special points called *roots of unity modulo p*. A number ω is an n-th root of unity mod p if $\omega^n \equiv 1 \pmod{p}$.

The project uses $p = 7340033 = 7 \times 2^{20} + 1$. Because $p - 1$ is divisible by large powers of 2, primitive n-th roots of unity exist for all $n \leq 2^{20}$.

Forward NTT. The Cooley–Tukey butterfly processes the array in $\log_2 n$ stages. At each stage the array is split into independent groups of size ℓ, and each group applies:

$$u = a[i+j], \quad v = a[i+j+\ell/2] \cdot \omega^j \bmod p$$

$$a[i+j] = (u+v) \bmod p, \qquad a[i+j+\ell/2] = (u-v+p) \bmod p$$

Inverse NTT. A known identity gives $\mathrm{INTT}(a) = \frac{1}{n}\mathrm{rev}(\mathrm{NTT}(a))$, so the inverse simply runs a forward NTT, reverses indices $1 \ldots n-1$, and multiplies every element by $n^{-1} \bmod p$ (computed via Fermat's little theorem: $n^{-1} \equiv n^{p-2} \bmod p$).

Polynomial multiplication. The convolution theorem gives:

$$a * b = \mathrm{INTT}(\mathrm{NTT}(a) \odot \mathrm{NTT}(b))$$

reducing $O(n^2)$ multiplication to $O(n \log n)$.

Parallelism. Groups at each butterfly stage are mutually independent — no group touches another group's memory. This is the natural OpenMP target:

```
#pragma omp parallel for schedule(static) num_threads(nthreads)
for (int64_t g = 0; g < num_groups; g++) { /* butterfly */ }
```

All parallel regions are guarded with `#ifdef _OPENMP` so the code runs correctly in serial when OpenMP is unavailable.

2.2 NTT in STARK Proofs

STARKs (Scalable Transparent ARguments of Knowledge) require a prover to certify a computation without revealing private inputs. Proof generation reduces to three heavy polynomial operations, each implemented with NTT calls in this project.

Step 1 — Constraint polynomial. The computation is encoded as a *trace* — a sequence of values representing each step. For the demo, the trace is a Fibonacci sequence mod p. The constraint

$$C(x) = \mathrm{Trace}(x{+}2) - \mathrm{Trace}(x{+}1) - \mathrm{Trace}(x)$$

must equal zero at every valid step. $C(x)$ is multiplied by the *zerofier* (a polynomial that vanishes at all trace positions) to produce the full constraint polynomial. A verifier checks divisibility without seeing the trace.

Step 2 — DEEP-FRI quotient. A random out-of-domain point z is chosen. The quotient

$$Q(x) = \frac{C(x)}{x - z}$$

shifts queries away from the evaluation domain, preventing a cheating prover from fabricating consistent answers. Polynomial division is done via NTT.

Step 3 — FRI folding. The FRI protocol iteratively halves the polynomial. At each round, $Q(x)$ is split into even and odd coefficients and folded with a random verifier challenge α:

$$\mathrm{FRI}(x) = Q_{\mathrm{even}}(x^2) + \alpha \cdot Q_{\mathrm{odd}}(x^2)$$

Each fold halves the problem size. NTT is called at every fold. In practice, a zkEVM proof may run millions of NTT operations per second — making NTT latency the dominant cost in proof generation.

3 Why a Learned Cost Model

Spawning threads has overhead. For small NTT sizes this overhead exceeds the computation itself, making maximum threads several times *slower* than a single thread. The crossover point depends on input size and hardware.

The project trains an XGBoost regressor on benchmark measurements across all combinations of NTT size and thread count. Given a new size, the model scores 13 candidate thread counts and recommends the fastest. Key design choices:

- **Predict runtime, not thread count.** Regression on a smooth target generalises better than classification on arbitrary labels.
- **Feature:** $\log_2(n)$**.** NTT sizes span four orders of magnitude; the log distributes them evenly.
- **Target:** $\log(1 + t_{\mu s})$**.** Compresses the range so large-n cases do not dominate the loss.

XGBoost achieves 100% exact-match accuracy vs 66.7% for an MLP, because the optimal thread count is a *step function* of n — exactly the structure that decision-tree splits represent natively.

4 Results

4.1 Speedup Curve

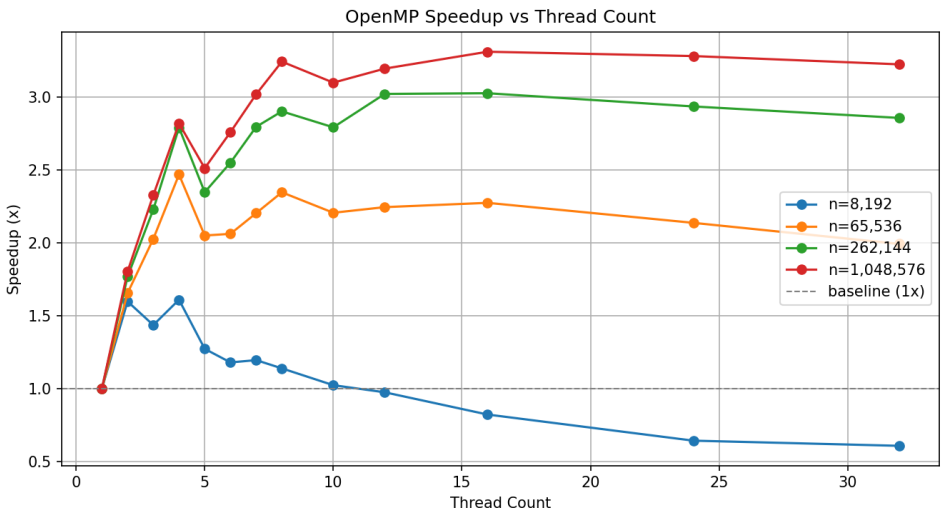

Figure 1: Parallel speedup over serial baseline. For $n = 1\,048\,576$ the NTT runs $\sim$3.3$\times$ faster at the sweet-spot thread count.

4.2 Threading Overhead Crossover

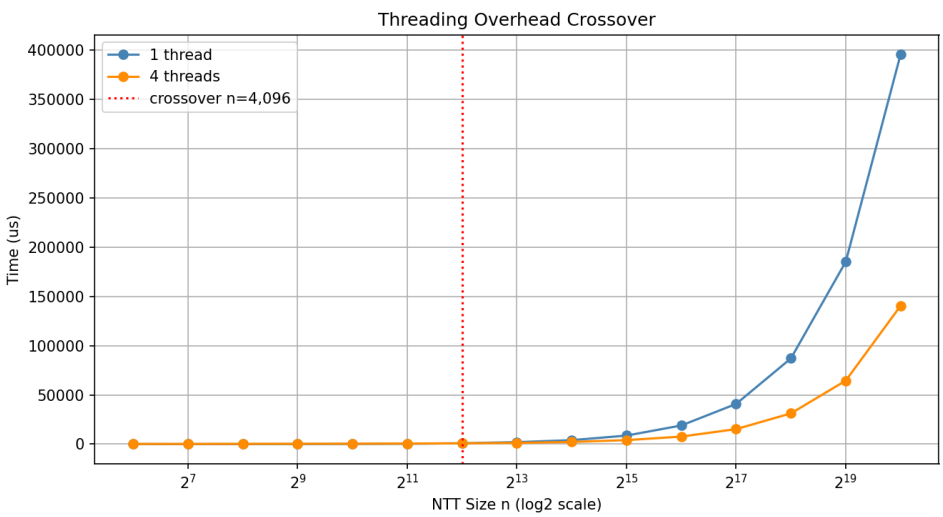

Figure 2: Raw wall-clock time for 1 thread vs 4 threads. The lines cross at $n = 4096$ — below this, 1 thread is faster.

4.3 MLP Training Loss

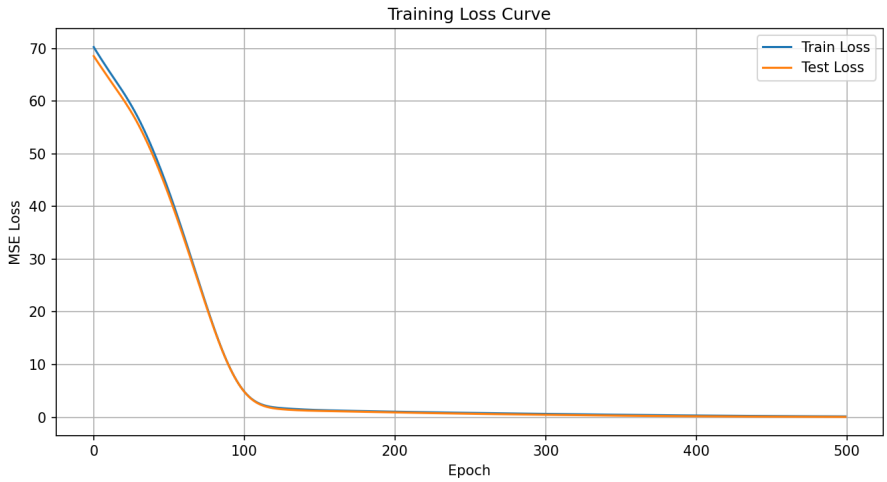

Figure 3: MLP train and test loss over 500 epochs. Loss drops from ∼71 to near 0 by epoch 300 with no overfitting.

4.4 MLP: Predicted vs Actual Thread Count

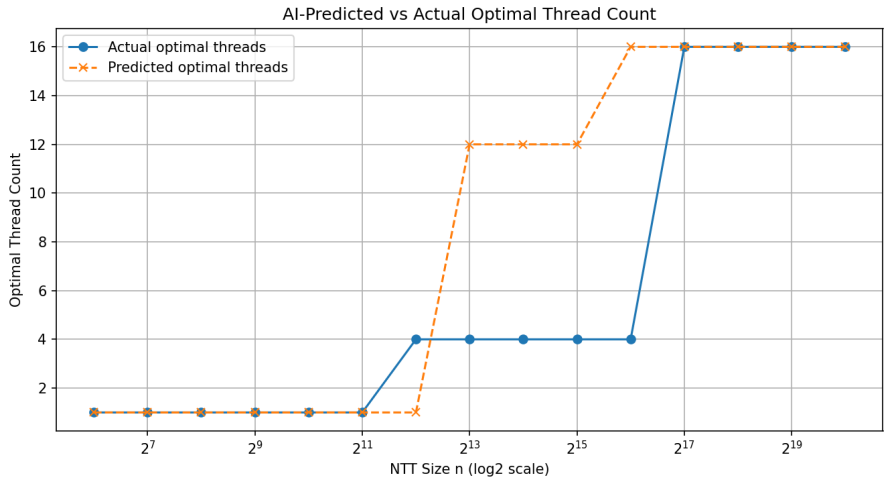

Figure 4: MLP predictions on the held-out test set. Exact-match accuracy: 46.7%. The model smooths over sharp regime boundaries.

4.5 XGBoost Feature Importance

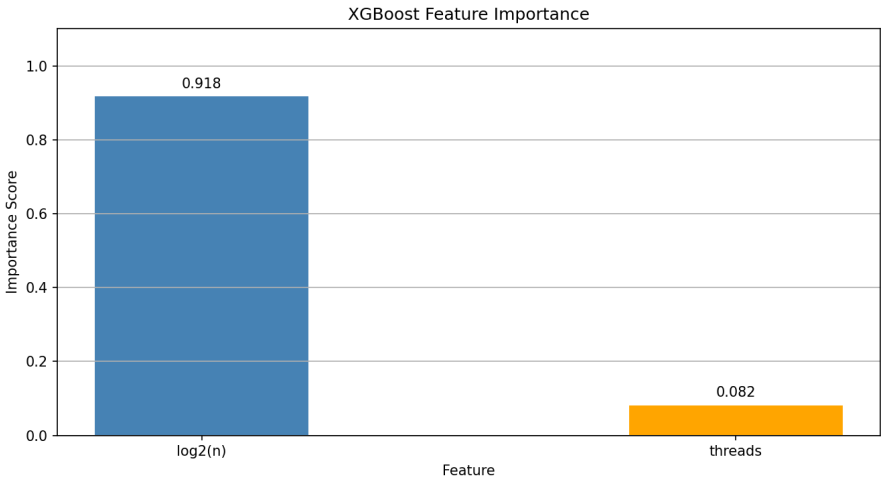

Figure 5: $\log_2(n)$ accounts for 91.8% of the XGBoost decision — thread count is almost entirely determined by NTT size.

4.6 XGBoost: Predicted vs Actual Thread Count

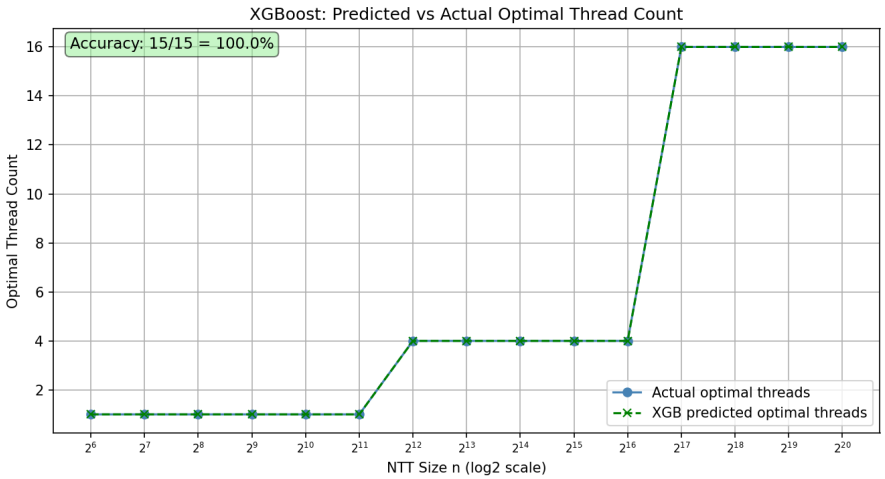

Figure 6: XGBoost achieves 100% exact-match accuracy across all 15 NTT sizes.

4.7 Three-Way Comparison: Serial vs Pure-HPC vs AI-HPC

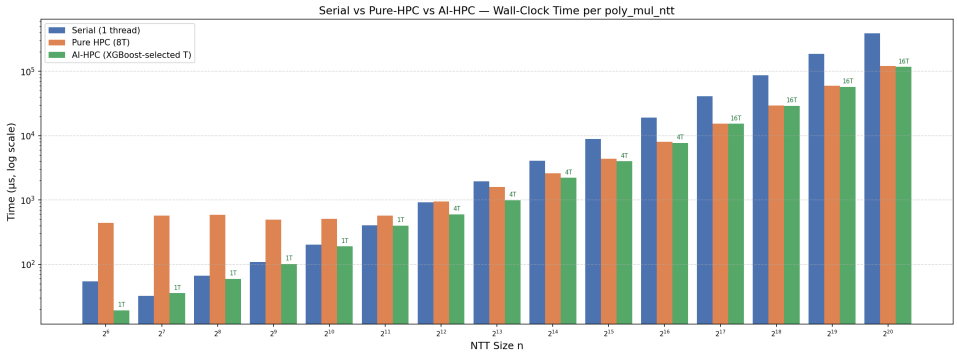

Figure 7: Wall-clock time across all sizes. Pure-HPC is up to 17× slower than serial for small n; AI-HPC avoids this entirely.

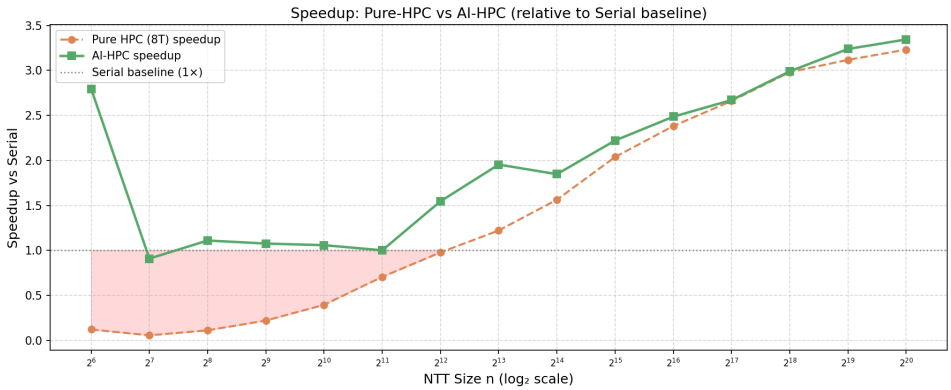

Figure 8: Speedup relative to serial. The red zone shows where Pure-HPC regresses below 1×. AI-HPC stays above the baseline throughout.

4.8 Key Numbers

n	Serial (μs)	Pure-HPC (μs)	AI-HPC (μs)	AI Threads	Speedup
64	53.8	440.5	19.2	1	2.80×
4 096	923.2	943.7	597.1	4	1.55×
65 536	19 194	8 056	7 720	4	2.49×
1 048 576	392 771	121 531	117 444	16	3.34×

Table 1: Three-way comparison at selected NTT sizes.

4.9 STARK Demo Output

```
  Prime modulus P = 7340033  (7 * 2^20 + 1)
  Trace length    = 4096
  Sequence        = Fibonacci mod P

  OpenMP available — max threads = 8

  Pipeline:
    Step 1  Constraint poly  C(x) = Trace(x) * Zerofier(x)
    Step 2  Quotient poly    Q(x) = C(x) * (x - z)   [DEEP-FRI]
    Step 3  FRI fold         FRI(x) = Q_even + alpha * Q_odd

  AI thread map (XGBoost, from trained model):
    NTT size <= 2048   → 1 thread
    NTT size <= 65536  → 4 threads
    NTT size >= 131072 → 16 threads

------------------------------------------------------------
  Correctness (all modes must match)
------------------------------------------------------------
  Step 1 C(x)[0]  = 1   PASS
  Step 2 Q(x)[0]  = 7339991   PASS
  Step 3 FRI[0]   = 3422866   PASS

------------------------------------------------------------
  Wall-clock time (microseconds)
------------------------------------------------------------
Mode          Step1(μs)  Step2(μs)  Step3(μs)    Total(μs)
-------------------------------------------------------------
Serial            3668.3      4301.2      1761.1        9730.6
Pure-HPC          2757.2      2683.0      1418.4        6858.6
AI-HPC            1664.7      1725.5      1082.8        4473.0

------------------------------------------------------------
  Speedup vs Serial
------------------------------------------------------------
Mode               Step1       Step2       Step3         Total
-------------------------------------------------------------
Serial             1.00x       1.00x       1.00x         1.00x
Pure-HPC           1.33x       1.60x       1.24x         1.42x
AI-HPC             2.20x       2.49x       1.63x         2.18x

------------------------------------------------------------
  AI-HPC thread selection (XGBoost decisions)
------------------------------------------------------------
  Step 1  NTT size =     8192  →  4 threads
  Step 2  NTT size =    16384  →  4 threads
  Step 3  NTT size =     8192  →  4 threads

  Key insight: Step 3 NTT is small (q_odd is half-length).
  Pure-HPC spawns max threads for it — paying overhead.
  AI-HPC uses 1 thread there — no unnecessary cost.
```

Figure 9: Output of `stark_demo.cpp`. Three STARK steps are timed in Serial / Pure-HPC / AI-HPC modes. AI-HPC achieves 2.18× overall speedup vs 1.42× for Pure-HPC by selecting 4 threads for Step 3 (NTT size 8192) instead of spawning the full thread pool.

5 Conclusion

Blindly maximising thread count is not an HPC strategy — it is a regression for small inputs. By training an XGBoost cost model on benchmark data, AI-HPC selects the right thread count at runtime and outperforms both the serial baseline and naive maximum-thread parallelism across the full NTT size range. Applied to a simulated STARK proof workload, the approach delivers a 2.18× end-to-end speedup.